UNIVERSIDADE DA CORUÑA

FACULTAD DE INFORMÁTICA
GRADO EN INTELIGENCIA ARTIFICIAL

Aprendizaje Profundo
Práctica 3: Un simulador neuronal de CarRacing
Curso 2026/2027

Instrucciones y calendario

Construye un **simulador neuronal de CarRacing**: dadas doce imágenes recientes y 35 acciones discretas (11 pasadas y 24 futuras), genera las 24 imágenes siguientes. No tienes que aprender a conducir ni utilizar aprendizaje por refuerzo.

Trabaja con **Keras**, Jupyter Notebooks y las imágenes RGB de 48×48 del entorno `AP-CarRacing48-v1`. El material de la práctica se facilitará en el Campus Virtual. Los datos de evaluación están disponibles en la competición.

Acceso a Kaggle: AP 2026 — CarRacing World Models (enlace de invitación).

Dependencias específicas: `gymnasium[box2d]` para el entorno (incluye Box2D para la física y Pygame para el renderizado) y `scikit-image` para SSIM. Instala el fichero proporcionado en una celda de Colab con `%pip install -r requirements.txt`. Para el comparador de la parte 5, instala también `requirements-interactive.txt` (`ipywidgets`, `anywidget` y `Pillow`). Mantén los módulos proporcionados en el directorio del notebook.

Parte	Tema	Fecha límite (viernes)	Semana de defensa
1	Autoencoders	20 de noviembre de 2026	23–27 de noviembre
2	Redes residuales	27 de noviembre de 2026	30 de noviembre–4 de diciembre
3	Redes recurrentes	4 de diciembre de 2026	7–11 de diciembre
4	Transformers	11 de diciembre de 2026	14–18 de diciembre
5	Juego y competición	18 de diciembre de 2026	Sin defensa en clase

Las defensas de las partes 1–4 se realizarán en clase durante la semana siguiente a su fecha límite, en la sesión correspondiente a cada grupo. La parte 5 se evaluará mediante la entrega, sin defensa en clase.

Criterios de evaluación

Cada parte vale **2 puntos**: 0,8 por implementación y resultados reproducibles, 0,6 por comparación e interpretación y 0,6 por defensa individual. En la parte 5, los 0,6 puntos de defensa se evaluarán mediante la explicación de las decisiones y las pruebas incluidas en la entrega. La práctica suma un máximo de **10 puntos**.
En las partes 1–4, cada retraso en la defensa reduce en **0,2 puntos el máximo de esa parte**, hasta un límite mínimo de 1 punto. Se aplica un corte, no una resta a la nota obtenida:

$$L_i = \min\{C_i, \max\{1, 2 - 0{,}2 r_i\}\}, \qquad N = \sum_{i=1}^{5} L_i.$$

C_i: calificación sobre 2; r_i: número de retrasos; L_i: nota final de la parte; N: nota total. El mínimo de 1 se refiere al límite, no a una nota garantizada.
Ejemplo: dos retrasos dejan el máximo en 1,6. Una calificación de 1,6 se mantiene; una de 1,8 queda en 1,6; una de 0,8 sigue siendo 0,8.
En la defensa explicarás tus decisiones y justificarás o realizarás un cambio breve. Se permite IA durante el desarrollo, declarando su uso; la defensa será sin IA. Se valorará el análisis aunque una modificación no mejore el resultado. El puesto en Kaggle no añade puntos.

Entrega común a todas las semanas

Mantén el grupo de las prácticas anteriores. Entrega un notebook ejecutado, código auxiliar, configuración, dependencias y modelos `.keras`, con los nombres de todos los integrantes. Conserva una versión de cada entrega e incluye resultados visuales, tiempos y una breve conclusión. Declara la ayuda externa y las herramientas de IA utilizadas.

Qué entra y qué sale del modelo

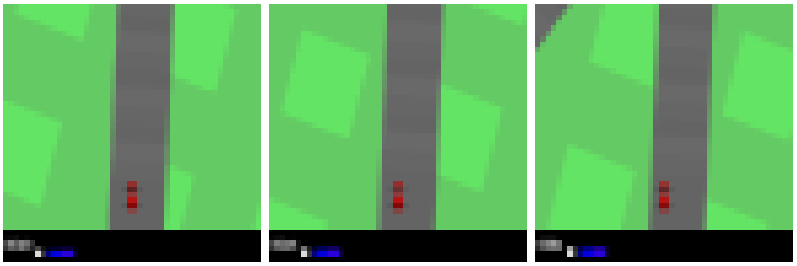


Figura 1: Capturas reales del simulador a 48×48, ampliadas para ver los píxeles.
Son ejemplos de observaciones, no predicciones de una red.



Lectura del diagrama. Este esquema representa el sistema completo de las partes 3–5. En las partes 1 y 2, el autoencoder recibe únicamente una imagen y reconstruye esa misma imagen; las acciones se incorporan al predictor temporal a partir de la parte 3. B es el tamaño del lote. Las imágenes usan canales al final, como en la configuración habitual de Keras. Normaliza los píxeles de $[0, 255]$ a $[0, 1]$. Cada acción es un entero: **0 = nada, 1 = derecha, 2 = izquierda, 3 = acelerar, 4 = frenar**. No son magnitudes continuas ni acciones simultáneas. Codifícalas con *one-hot* de cinco componentes y proyéctalas con una capa lineal aprendida; los identificadores no representan una escala numérica.

La acción a_t lleva de x_t a x_{t+1}. Cada paso de nuestra variante repite esa acción durante cuatro pasos físicos. Las 24 imágenes futuras cubren aproximadamente **1,92 segundos**.

Reglas comunes a todos los experimentos

1. **Datos:** genera entrenamiento y validación con semillas distintas mediante `generate_sequences(seed, n_sequences)`. Cada secuencia larga procede de un circuito independiente. Separa entrenamiento y validación antes de crear ventanas: todas las ventanas de un circuito deben permanecer en la misma partición. Los ejemplos reservados de Kaggle no se usarán para entrenar ni para seleccionar hiperparámetros.
2. **Sin mirar el futuro:** en evaluación solo puedes usar las doce imágenes iniciales, las acciones y tus propias predicciones. El predictor no puede consultar imágenes futuras reales ni obtenerlas ejecutando el simulador. En la parte 5, la interfaz sí ejecuta el entorno real como referencia visual independiente, sin alimentar con él al modelo. No se permite usar el simulador para reconstruir los objetivos de Kaggle.
3. **Mismas condiciones:** compara sobre los mismos ejemplos y registra el coste. Conserva semillas y modelos. No utilices imágenes de mayor resolución, estado físico interno, recompensas como entrada ni modelos externos preentrenados.
4. **Resultados visuales:** muestra realidad, predicción y error. Incluye aciertos y fallos; el modelo no puede conocer exactamente un tramo del circuito que aún no ha visto.

Trabajo a realizar

Cada semana empieza en una página nueva. Consulta solo su bloque de tareas y los anexos indicados; conserva los datos y modelos para la semana siguiente.

1. Autoencoder — Entrega: 20 de noviembre

Objetivo. Reconstruir imágenes individuales sin acciones y estudiar la compresión.

Ficheros necesarios. `dataset.py`, `carracing48.py`, `metric.py`, `requirements.txt` y `DATOS.md`.

Qué reutilizar. Genera y guarda las particiones por circuito y las semillas; se reutilizarán durante toda la práctica.

Entrada y objetivo: solo imágenes, sin acciones. Cada ejemplo es un fotograma RGB x_t de 48×48, normalizado a $[0, 1]$. El codificador lo comprime en un vector z_t y el decodificador intenta reconstruir el mismo fotograma:

$$x_t \xrightarrow{\text{codificador}} z_t \in \mathbb{R}^d \xrightarrow{\text{decodificador}} \tilde{x}_t, \qquad \mathcal{L} = \text{MSE}(x_t, \tilde{x}_t).$$

Entrada y objetivo tienen forma $(B, 48, 48, 3)$; el latente, (B, d). No se introduce una secuencia de doce imágenes ni se predice x_{t+1}. Puedes usar todos los fotogramas de las secuencias largas de entrenamiento como fotogramas independientes; puedes barajarlos, conservando la separación por circuitos respecto a validación y test.

1. Inspecciona una secuencia y comprueba imágenes, acciones y alineación temporal. Las acciones se inspeccionan para conocer los datos, pero no se suministran al autoencoder. Crea, por separado y sin entrenamiento, la referencia que repite la última imagen durante 24 pasos; servirá para comparar los predictores de las semanas siguientes.
2. Entrena un autoencoder convolucional sin conexiones residuales. Compara dos tamaños del espacio latente manteniendo el resto del experimento constante.
3. Muestra originales, reconstrucciones y mapas de error; calcula MSE en validación. Guarda el modelo.

Funciones proporcionadas: importa `generate_sequences` de `dataset` para obtener las imágenes; usa semillas distintas para entrenamiento y validación. `visual_metrics(originales, reconstrucciones)` devuelve, entre otros valores, `mse`. Para explorar el juego, usa `make_env()` de `carracing48` y sus métodos `reset`, `step` y `render`.

Apoyo en Keras: `Conv2D`, `Conv2DTranspose` o `UpSampling2D`. Entrena con `autoencoder.fit(X_train, X_train, ...)` y valida con el par `(X_val, X_val)`. Conserva codificador y decodificador para reutilizarlos por separado.
En la defensa: dimensiones, información perdida y diferencia entre reconstruir y predecir.

Qué entregar. Notebook ejecutado; comparación de dos tamaños latentes; reconstrucciones, mapas de error y MSE; codificador y decodificador guardados.

2. Autoencoder residual — Entrega: 27 de noviembre

Objetivo. Evaluar el efecto de las conexiones residuales y elegir la representación visual.

Ficheros necesarios. `metric.py`; datos, notebook y modelos de la semana 1. `dataset.py` y `carracing48.py` solo si necesitas generar más datos.

Qué reutilizar. Usa la misma partición y las mismas imágenes de evaluación de la semana 1.

1. Incorpora al menos dos bloques residuales, con dos convoluciones por rama aprendida. Asegura que las dimensiones permiten sumar entrada y salida.
2. Compara las conexiones de salto activadas y desactivadas con las mismas ramas, datos y presupuesto. Informa de parámetros, curvas y reconstrucciones.
3. Elige el autoencoder que usarás en las partes 3 y 4. Congélalo y precalcula los latentes de cada partición por separado.

Función proporcionada: usa `visual_metrics(originales, reconstrucciones)` de `metric` para comparar ambos autoencoders sobre las mismas imágenes guardadas en la parte 1.

Apoyo en Keras: API funcional, `Add` y `trainable=False`; recompila el modelo de entrenamiento tras cambiar qué capas son entrenables.
En la defensa: suma residual, cambios de dimensiones y justificación del modelo elegido.

Qué entregar. Notebook con comparación controlada; encoder y decoder elegidos en formato .keras; latentes guardados por partición y justificación de la elección.

3. Ventanas y predictor recurrente — Entrega: 4 de diciembre

Objetivo. Predecir el siguiente latente con una LSTM condicionada por acciones y generar 24 imágenes.

Ficheros necesarios. `metric.py`, `DATOS.md`; secuencias y acciones guardadas, encoder y decoder de la semana 2. Consulta los anexos B y C.

Qué reutilizar. Conserva congelado el autoencoder y usa las particiones anteriores.

1. **Implementa el fraccionamiento** de las secuencias largas, con paso entre ventanas configurable. Para una ventana que empieza en s, usa como contexto $x_s, \ldots, x_{s+11}$, como objetivos $x_{s+12}, \ldots, x_{s+35}$ y como acciones $a_s, \ldots, a_{s+34}$. Las acciones futuras empiezan en a_{s+11}. Con longitud T y paso 1 hay $T - 35$ ventanas válidas (61 si $T = 96$). Descarta ventanas incompletas y nunca cruces episodios.

2. Comprueba formas, alineación del primer objetivo y ausencia de circuitos compartidos entre entrenamiento y validación. Aplica el mismo fraccionamiento a los latentes si los has precalculado.

3. **Predice en el espacio latente con una LSTM.** Sea E el codificador y D el decodificador elegidos en la parte 2. Para cada imagen, calcula $z_t = E(x_t)$. Transforma la acción discreta con una capa lineal aprendida: $e_t = W_a\,\text{onehot}(a_t) + b_a$. La entrada temporal es $u_t = [z_t; e_t]$ (concatenación), o bien $u_t = z_t + e_t$ si ambos vectores tienen dimensión d. Una capa lineal sobre la salida de la LSTM debe producir $\hat{z}_{t+1} \in \mathbb{R}^d$; decodifica con $\hat{x}_{t+1} = D(\hat{z}_{t+1})$.

4. Procesa los 11 pares históricos $(z_s, a_s), \ldots, (z_{s+10}, a_{s+10})$ y después (z_{s+11}, a_{s+11}) para predecir el primer objetivo x_{s+12}. Continúa hasta obtener 24 predicciones, usando los latentes predichos y las acciones siguientes. Mantén congelados E y D y entrena la proyección de acciones, la LSTM y la proyección de salida con la pérdida perceptual del anexo C.

5. Compara entrenamiento a un paso y a varios pasos. Evalúa realimentando predicciones; representa el error en horizontes 1–24 y compara con repetir la última imagen. Implementa también una operación de **predicción de un solo paso** que reciba únicamente el historial y la acción actual: se usará en el juego de la parte 5.

Funciones proporcionadas: `generate_sequences` devuelve `frames` y `actions`, sin fraccionar. Debes implementar el ventaneo. Evalúa los objetivos y predicciones con `visual_metrics`; usa la misma función por horizonte.

Apoyo: consulta el esquema y la pérdida perceptual del anexo C. Usa `LSTM`, `Dense`, `Concatenate` o `Add`.

En la defensa: diferencia entre usar el dato real anterior durante el entrenamiento (*teacher forcing*) y utilizar la predicción propia en evaluación.

Qué entregar. Código de ventaneo; predictor LSTM de un paso en .keras; bucle de 24 pasos; comparación de entrenamiento a uno/varios pasos, curvas por horizonte y vídeos.

4. Transformer — Entrega: 11 de diciembre

Objetivo. Comparar la LSTM con un transformer causal que predice en el mismo espacio latente.

Ficheros necesarios. `metric.py`; ventaneo, datos, pérdida perceptual y modelos de las semanas 2 y 3. Consulta los anexos B y C.

Qué reutilizar. Mantén el autoencoder, las particiones y la evaluación para que la comparación sea justa.

1. Sustituye la LSTM por un transformer causal pequeño que también prediga $\hat{z}_{t+1}$. Construye cada token a partir de z_t y de la proyección lineal de onehot(a_t), concatenándolos o sumándolos como en la parte 3; adapta la dimensión del token y añade información posicional. El token de tiempo t puede ver los pares hasta (z_t, a_t), pero nunca z_{t+1} ni acciones posteriores a a_t.
2. Conserva las ventanas, el codificador y el decodificador congelados y la misma pérdida perceptual sobre $D(\hat{z}_{t+1})$ de la parte 3. Evalúa 24 pasos realimentando latentes predichos. Permite también avanzar un solo paso con el historial y la acción actual, sin disponer de acciones futuras.
3. Verifica que cambiar imágenes o acciones posteriores a un instante no altera las predicciones de ese instante, con el modelo en modo inferencia. Compara con la recurrente: error por horizonte, vídeos, parámetros y tiempo.
4. Reentrena una variante sin información de las acciones y analiza qué cambia.

Función proporcionada: reutiliza los datos de la parte 3 y `visual_metrics(objetivos, predicciones)` para mantener la misma evaluación. La máscara causal y su prueba debes implementarlas tú.

Apoyo en Keras: Usa la implementación de un transformer introducida en los notebooks de la asignatura. Usa una configuración de pocas capas y tamaño reducido del *embedding*
En la defensa: qué información puede ver cada token y cómo detectar una fuga del futuro.

Qué entregar. Transformer de un paso en .keras; prueba de causalidad; comparación con LSTM y con la variante sin acciones; vídeos, parámetros y tiempos.

5. Entorno real frente al modelo — Entrega: 18 de diciembre

Objetivo. Comparar la predicción conjunta de 24 instantes con la predicción secuencial y comprobar esta última en un juego interactivo.

Ficheros necesarios. `interactive_alumno.py`, `comparador.ipynb`, `INTERACTIVO.md`, `requirements-interactive.txt`, `carracing48.py`, `metric.py`, `submission.py`, `test_inputs.npz` y `sample_submission.csv`.

Qué reutilizar. Conserva congelados el encoder y decoder elegidos y reutiliza uno de los predictores temporales entrenados. Entrena fuera del juego una variante de salida conjunta, manteniendo los mismos datos y la pérdida perceptual del anexo C.

Tareas.

1. **Predicción conjunta frente a secuencial.** Implementa y compara dos estrategias sobre las mismas ventanas de validación:

   - *Conjunta:* a partir de los 12 latentes iniciales y las acciones, produce los 24 latentes futuros en una sola salida $(B, 24, d)$, sin realimentar las predicciones. Decodifícalos para obtener las 24 imágenes.
   - *Secuencial (autorregresiva):* predice un latente (B, d), añádelo al historial, elimina el más antiguo y repite con la siguiente acción hasta completar los 24 pasos. A partir del primer paso, el dato anterior que se introduce es el **latente predicho**, no el real ni una imagen recodificada.

   Para cada horizonte h, ambas estrategias solo pueden utilizar las acciones hasta la que produce x_{t+h}, nunca posteriores; aplica la restricción también en la variante conjunta. Recibir las 35 acciones juntas no permite usarlas indiscriminadamente en todas las salidas. Una llamada que ejecuta internamente un bucle autorregresivo sigue siendo secuencial: la diferencia está en la dependencia de las predicciones, no en el número de llamadas a Python.

2. **Comparación controlada.** Conserva particiones, autoencoder, características perceptuales y presupuesto de entrenamiento; informa de las diferencias de parámetros. Compara puntuación por horizonte (1–24), vídeos, tiempo total para generar 24 imágenes y memoria si puedes medirla. Entrena o ajusta la variante secuencial con pérdida sobre varios pasos para compararla con la conjunta. Si utilizas el latente real anterior durante el entrenamiento (*teacher forcing*), declárralo; en evaluación nunca se suministra. Analiza acumulación de errores, coherencia temporal y coste, sin asumir que una estrategia será mejor.

3. Lee la clase `ComparisonSession`, especialmente `reset` y `step`. Identifica los tres métodos pendientes y explica qué datos necesita cada uno. Completa la inicialización del estado latente, la inferencia de un paso y la actualización de los historiales. Mantén las entradas y salidas indicadas para cada método; no modifiques los controles ni el simulador para eludir la integración.

4. Codifica las doce imágenes iniciales una sola vez. Después realimenta directamente los latentes predichos; el decoder solo produce la imagen visible. Alinea las once acciones históricas y la acción actual, conserva los tamaños de las ventanas y ejecuta los modelos en modo inferencia. Nunca consultes imágenes reales nuevas ni el estado interno del entorno para predecir.

5. Diseña comprobaciones que detecten errores de reinicio, desalineación de acciones y fuga de imágenes reales. Demuestra también que no vuelves a codificar las imágenes generadas. Explica por qué cada prueba detectaría una implementación incorrecta.

6. Ejecuta `comparador.ipynb` con tu clase y modelos. Juega al menos 48 pasos con WASD, registra acciones y frecuencia efectiva y analiza la divergencia. Reproduce una partida de 24 pasos desde la misma semilla y calcula sus métricas.

7. Elige la estrategia para Kaggle usando únicamente validación, justifica la elección y guarda sus pesos. Para la variante secuencial, comprueba que el bucle de 24 pasos coincide con 24 llamadas consecutivas al predictor de un paso, salvo tolerancia numérica. No se exige que las predicciones de la variante conjunta coincidan con las secuenciales.

Acciones en el juego. Usa la variante secuencial: el jugador decide cada acción en ese momento. La comparación con la conjunta se hace sobre secuencias grabadas, donde ambas disponen de las mismas acciones; no inventes 24 acciones futuras para conectar la salida conjunta al control en vivo.

Ayuda proporcionada. La plantilla indica qué recibe y qué debe devolver cada método; `launch_comparison(encoder=..., dynamics=..., decoder=...)` arranca la interfaz cuando la clase esté completada. Usa `visual_metrics` para

evaluar y `write_submission` para exportar. Consulta controles y formas de los tensores en el anexo B y en `INTERACTIVO.md`.

Explicación en la entrega (sin defensa en clase). Explica la diferencia entre predicción conjunta, autorregresión y teacher forcing, y los puntos de integración. Incluye una demostración grabada del juego y resultados que muestren cómo tus pruebas detectan un fallo introducido en la actualización temporal.

Qué entregar. Clase completada, notebook con la comparación de ambas estrategias y demostración del juego, encoder, decoder y ambos predictores en .keras, curvas por horizonte, tiempos, pruebas de integración, partida registrada y CSV de Kaggle.

Anexo A. Ficheros proporcionados

carracing48.py Entorno de 48×48; `make_env()` permite explorarlo.

dataset.py `generate_sequences(seed, n_sequences)` genera secuencias largas de imágenes y acciones; debes fraccionarlas en ventanas.

metric.py `visual_metrics(target, prediction)` calcula puntuación, SSIM, error de bordes y MSE. `score` adapta la evaluación al CSV de Kaggle.

submission.py `write_submission(predictions, sequence_ids, path)` guarda el CSV de envío.

interactive_alumno.py Clase `ComparisonSession` con tres métodos pendientes; incluye los controles y paneles del juego.

comparador.ipynb Notebook para probar la clase completada con tus modelos preentrenados.

INTERACTIVO.md Entradas y salidas de los modelos y uso de los controles; dependencias en `requirements-interactive.txt`.

requirements.txt Dependencias y versiones para instalar el material.

DATOS.md Ejemplos de generación, formatos y alineación de acciones e imágenes.

test_inputs.npz Contextos, acciones e identificadores reservados de Kaggle, sin objetivos.

sample_submission.csv Ejemplo válido de envío que repite la última imagen.

Las funciones de métricas reciben imágenes normalizadas a $[0, 1]$. Los modelos y sus bucles de predicción debes implementarlos tú.

Anexo B. Formatos de datos y funcionamiento del juego

Datos que devuelve el generador. Por defecto devuelve `frames` $(N, 96, 48, 48, 3)$ y `actions` $(N, 95)$: 96 fotogramas consecutivos por circuito y 95 acciones enteras. La acción a_j transforma x_j en x_{j+1}. Antes de registrar, ejecuta y descarta un número de observaciones elegido uniformemente entre 0 y 200, ambos incluidos, aplicando la misma política de conducción. Cada observación equivale a 0,08 s simulados; así las secuencias pueden comenzar con el coche ya en movimiento. No entrega ventanas ni pares entrada–objetivo; construirlos forma parte de la práctica. Puedes ajustar la longitud mediante `sequence_length` (36–160).

Datos proporcionados en Kaggle. Recibirás ventanas ya preparadas: `context` $(N, 12, 48, 48, 3)$, `actions` $(N, 35)$ e identificadores. Las primeras 11 acciones unen los fotogramas del contexto; las 24 restantes producen los objetivos reservados $(N, 24, 48, 48, 3)$. Las secuencias largas que originaron estas ventanas no se publican.

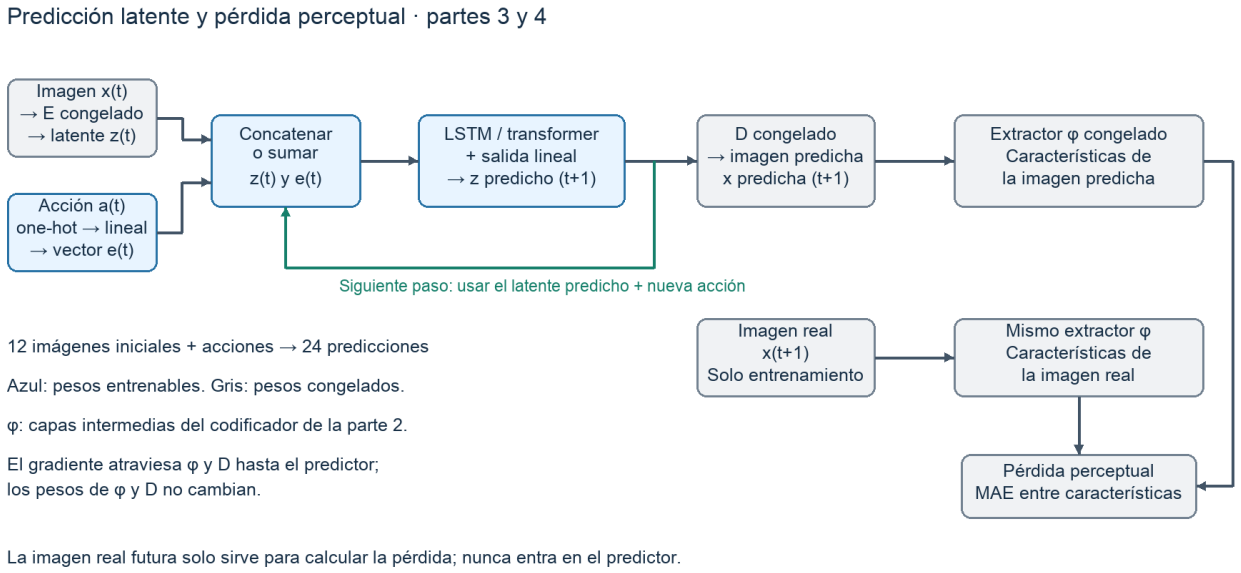
Keras: puedes usar la API funcional, capas estándar y `Model.fit`. Separa codificador, predictor temporal y decodificador para poder reutilizarlos. Guarda los modelos en formato `.keras`, incluyendo el código necesario para cargar cualquier capa personalizada.

Modelos de un paso. Encoder: $(B, 48, 48, 3) \to (B, d)$. Predictor temporal: lista con latentes $(B, 12, d)$, acciones históricas $(B, 11)$ y acción actual $(B,)$; salida (B, d). Decoder: $(B, d) \to (B, 48, 48, 3)$ en $[0, 1]$. Acciones enteras 0–4; la proyección lineal pertenece al predictor. Usa modelos sin estado persistente oculto (`stateful=False`); el historial viaja en cada llamada.

Predicción de Kaggle. Entrega una función que reciba contextos $(B, 12, 48, 48, 3)$ y acciones $(B, 35)$ y devuelva $(B, 24, 48, 48, 3)$ en $[0, 1]$, con instrucciones para ejecutarla sin reentrenar. Inicializa los latentes una vez. En la variante secuencial actualiza su ventana con cada predicción; en la conjunta genera los 24 latentes sin realimentación. Ambas deben respetar la información disponible en cada horizonte.

Controles del juego. Pulsa Jugar y mantén enfocado el campo de control: W acelera, S frena y A/D giran. Sin teclas se aplica nada; con varias, manda la última mantenida. Escape o perder el foco pausa. El objetivo es 24 actualizaciones/s; si el modelo o la conexión son lentos, la frecuencia baja sin acumular órdenes. Cada paso equivale a 0,08 s simulados: a 24 Hz la simulación avanza a 1,92×. Los botones permiten diagnosticar paso a paso; no se penaliza una frecuencia menor si se registra y explica.

Anexo C. Predictor latente y pérdida perceptual



Pérdida de entrenamiento (partes 3 y 4). No minimices MSE entre latentes. Compara la imagen decodificada con el fotograma real siguiente mediante características visuales:

$$\mathcal{L}_{\text{perc}}(\hat{x}_{t+1}, x_{t+1}) = \frac{1}{|\mathcal{J}|} \sum_{\ell \in \mathcal{J}} \frac{\|\phi_\ell(D(\hat{z}_{t+1})) - \phi_\ell(x_{t+1})\|_1}{n_\ell}.$$

Aquí ϕ_ℓ es la activación de una capa convolucional del codificador congelado de la parte 2 y n_ℓ su número de elementos por imagen. Selecciona al menos dos capas a distintas profundidades, anteriores al vector latente, y documenta cuáles usas. Conserva las mismas capas en ambas partes. El objetivo es la imagen real, no su reconstrucción. Promedia sobre el lote y, si entrenas a varios pasos, sobre los horizontes utilizados.

Congelar no es cortar el gradiente. Los pesos del decodificador y del extractor de características no se actualizan, pero el gradiente debe atravesarlos desde la pérdida hasta el predictor. Ejecuta estos modelos en modo inferencia para fijar también BatchNorm y Dropout. No uses `predict()`, NumPy ni `stop_gradient` en la rama de imágenes predichas durante el cálculo de la pérdida. Puedes precalcular los latentes y las características de los objetivos reales. `visual_metrics` se usa solo para evaluar; no implementa esta pérdida diferenciable.

Apoyo en Keras: `LSTM`, `Dense(activation=None)`, `Concatenate` o `Add`. Construye un modelo extractor con las salidas intermedias del codificador y aplica la pérdida con operaciones de Keras. Usa `trainable=False` y llamadas con `training=False` para los modelos congelados.

Anexo D. Evaluación y formato de Kaggle

Cómo se comparan los modelos

Kaggle evaluará **similitud estructural y bordes**, no únicamente distancia L2. Sobre imágenes RGB en $[0, 1]$, la puntuación será:

$$S = 0{,}8\,\frac{1 - \mathrm{SSIM}_{\mathrm{RGB}}}{2} + 0{,}2G, \qquad G = \frac{\mathrm{MAE}(D_x\hat{x}, D_x x) + \mathrm{MAE}(D_y\hat{x}, D_y x)}{4}.$$

SSIM usa ventana uniforme 7×7, rango 1 y covarianza muestral, promediando los canales RGB. D_x, D_y son diferencias entre vecinos sin relleno. Se promedian por igual imágenes y secuencias. **Menor es mejor; una reproducción exacta puntúa 0**. MSE se conserva como diagnóstico. Ninguna de estas métricas garantiza por sí sola coherencia física: inspecciona también los vídeos.

El archivo `metric.py` define la evaluación exacta y `submission.py` exporta las predicciones. La clasificación final usará circuitos privados. Cada fila identifica secuencia y horizonte y contiene 6912 valores RGB, en orden fila–columna–canal, finitos y en $[0, 1]$. `sample_submission.csv` repite la última imagen y no contiene los objetivos.